



NITTE
(Deemed to be University)

**NMAM INSTITUTE
OF TECHNOLOGY**

(An off-Campus Institution of NITTE (DEEMED TO BE UNIVERSITY),
MANGALORE)

Department of MCA

15-3-2026

TASK 1

Answer the following in detail

1.Explain the standard structure of XHTML and discuss different basic text markup tags with examples.

XHTML Structure and Basic Text Markup Tags

XHTML is a stricter version of HTML that follows XML rules such as proper nesting, lowercase tags, and closing all elements.

Basic Structure:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Page</title>
</head>
<body>
  <p>Sample text</p>
</body>
</html>
```

Basic Text Markup Tags:

- <h1> – <h6>: Headings
- <p>: Paragraph
- : Bold text
- <i>: Italic text
-
: Line break

Example:

`<p><b>Bold</b> and <i>Italic</i> text</p>`

2.Explain different types of lists and tables in XHTML with suitable examples.

Lists and Tables in XHTML

Types of Lists:

- **Ordered List ()** – numbered items
- **Unordered List ()** – bullet items
- **Definition List (<dl>)** – terms and definitions

Example:

```
<ol>
<li>Item 1</li>
<li>Item 2</li>
</ol>
```

Tables:

Used to display data in rows and columns using `<table>`, `<tr>`, `<th>`, `<td>`.

Example:

```
<table border="1">
<tr><th>Name</th><th>Age</th></tr>
<tr><td>John</td><td>20</td></tr>
</table>
```

3.Explain HTML forms and frames with appropriate examples.

HTML Forms and Frames

Forms:

HTML forms are used to collect user input. They use the <form> tag along with input elements.

Example:

```
<form>  
Name: <input type="text" />  
<input type="submit" value="Submit" />  
</form>
```

Frames:

Frames divide a webpage into multiple sections, each showing a different HTML document using <frameset> and <frame> (used in older HTML).

Example:

```
<frameset cols="50%,50%">  
<frame src="left.html" />  
<frame src="right.html" />  
</frameset>
```

4. Describe the levels of style sheets in CSS and explain the advantages of CSS in web design.

Levels of Style Sheets in CSS

1. **Inline CSS** – applied directly inside HTML tags

```
<p style="color: red;">Text</p>
```

2. **Internal CSS** – defined inside <style> in <head>

```
<style>  
p { color: blue; }  
</style>
```

3. **External CSS** – written in a separate .css file and linked

<link rel="stylesheet" href="style.css" />

Advantages of CSS

- Separates content and design
- Improves consistency across pages
- Reduces code repetition
- Makes website maintenance easier
- Enhances page loading speed

5. Explain various CSS selector forms and property value forms with examples.

CSS Selector Forms

1. Element Selector – selects HTML elements

p { color: red; }

2. Class Selector (.class) – selects elements with a class

.note { color: blue; }

3. ID Selector (#id) – selects a specific element

#header { font-size: 20px; }

4. Universal Selector (*) – selects all elements

* { margin: 0; }

Property Value Forms

- Color values: color: red;, color: #ff0000;
- Length values: font-size: 16px;, width: 50%;
- Multiple values: margin: 10px 20px;
- Keyword values: display: block;

6. Describe the CSS box model and explain the role of span and div tags in web page layout.

CSS Box Model

The CSS box model defines the layout of elements as a rectangular box consisting of:

- Content – actual text or data
- Padding – space around content
- Border – surrounds padding
- Margin – space outside the border

Example:

```
div {  
  margin: 10px;  
  padding: 5px;  
  border: 1px solid black;  
}
```

Role of <div> and

- <div>: Block-level element used to group large sections of content (takes full width)
- : Inline element used to style small parts of text

Example:

```
<div>This is a block</div>  
<p>This is <span style="color:red;">important</span> text</p>
```

- ✓ Box model controls spacing and layout
- ✓ <div> structures layout, styles inline content

7. Explain the new semantic elements introduced in HTML5 and their significance.

HTML5 Semantic Elements

Semantic elements clearly describe the meaning of content in a webpage.

Common Elements:

- `<header>` – top section of page
- `<nav>` – navigation links
- `<section>` – grouping of content
- `<article>` – independent content
- `<aside>` – side content
- `<footer>` – bottom section

Example:

```
<header>My Website</header>
<nav>Menu</nav>
<section>
  <article>Content here</article>
</section>
<footer>Copyright</footer>
```

Significance

- Improves readability and structure
- Helps search engines (SEO)
- Enhances accessibility
- Makes code easier to maintain

8. Discuss HTML5 input types, new form elements, and attributes with examples.

HTML5 Input Types, Elements, and Attributes

New Input Types:

- email, url, number, date, range

```
<input type="email" />
```

```
<input type="date" />
```

New Form Elements:

- <datalist> – provides suggestions
- <output> – displays result

```
<input list="browsers" />
```

```
<datalist id="browsers">
```

```
<option value="Chrome" />
```

```
</datalist>
```

New Attributes:

- placeholder – hint text
- required – mandatory field
- autofocus – focuses automatically

```
<input type="text" placeholder="Enter name" required autofocus />
```

9. Explain responsive web design and viewport concepts in HTML5.

Responsive Web Design & Viewport in HTML5

Responsive Web Design:

It is a technique to make web pages adapt to different screen sizes (mobile, tablet, desktop) using flexible layouts, images, and CSS media queries.

Example:

```
@media (max-width: 600px) {  
  body { background-color: lightblue; }
```

}

Viewport Concept:

Viewport controls how a webpage is displayed on different devices.

Meta Tag:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

- width=device-width → matches screen width
- initial-scale=1.0 → sets zoom level

10. Explain the Bootstrap framework and Bootstrap grid system with an example of a responsive webpage.

Bootstrap Framework

Bootstrap is a popular CSS framework used to create responsive and mobile-first websites quickly using pre-defined classes.

Bootstrap Grid System

It uses a 12-column layout to design responsive pages. Rows contain columns that adjust based on screen size (col-sm, col-md, col-lg).

Example:

```
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0/dist/css/bootstrap.min.css" />
```

```
<div class="container">
```

```
<div class="row">
```

```
<div class="col-md-6">Left</div>
```

```
<div class="col-md-6">Right</div>
```


</div>

</div>

11. Explain the overview, features, and uses of PHP in web development.

Overview of PHP

PHP (Hypertext Preprocessor) is a server-side scripting language used to develop dynamic and interactive web pages. It runs on the server and generates HTML.

Features of PHP

- Open-source and free
 - Platform independent
 - Easy to learn and embed in HTML
 - Supports databases like MySQL
 - Fast and efficient
-

Uses of PHP

- Creating dynamic websites
- Handling forms and user input
- Database operations (CRUD)
- Session and cookie management

12. Explain the general syntactic characteristics of PHP and discuss primitives, operations, and expressions with examples.

PHP Syntax Characteristics

- PHP code is written inside <?php ... ?>
- Statements end with ;
- Variables start with \$
- Case-sensitive for variables

Example:

```
<?php
```

```
$name = "John";
```

```
echo $name;
```

```
?>
```

Primitives (Data Types)

- Integer, Float, String, Boolean

```
$x = 10;    // integer
```

```
$y = 5.5;   // float
```

```
$msg = "Hi"; // string
```

Operations & Expressions

- Arithmetic: +, -, *, /
- Assignment: =
- Comparison: ==, !=

```
$a = 5 + 3;
```

```
if ($a == 8) {
```

```
    echo "Correct";
```

```
}
```

13. Describe control statements in PHP with suitable examples.

Control Statements in PHP

Control statements are used to control the flow of execution in a program.

Types:

1. **Conditional Statements** – if, else, switch

```
<?php
```

```
$age = 18;
```

```
if ($age >= 18) {  
    echo "Adult";  
} else {  
    echo "Minor";  
}  
?>
```

2. Looping Statements – for, while

```
<?php  
for ($i = 1; $i <= 3; $i++) {  
    echo $i;  
}  
?>
```

14. Explain arrays in PHP and discuss different types of arrays with examples.

Arrays in PHP

An array is a data structure used to store multiple values in a single variable.

Types of Arrays:

1. Indexed Array – uses numeric indexes

```
<?php  
$arr = array("A", "B", "C");  
echo $arr[0];  
?>
```

2. Associative Array – uses key-value pairs

```
<?php  
$age = array("John" => 20, "Sam" => 22);  
echo $age["John"];  
?>
```

3. Multidimensional Array – array within array

```
<?php
$data = array(
    array(1, 2),
    array(3, 4)
);
echo $data[0][1];
?>
```

15. Explain functions in PHP and demonstrate user-defined functions with parameters and return values.

Functions in PHP

A function is a block of code used to perform a specific task and can be reused multiple times.

User-Defined Function with Parameters & Return Value

```
<?php
function add($a, $b) {
    return $a + $b;
}
```

```
$result = add(5, 3);
echo $result;
?>
```

Explanation:

- function add(\$a, \$b) → parameters
- return → sends result back
- Function is called using add(5, 3)

16. Explain pattern matching using regular expressions in PHP with examples.

Pattern Matching using Regular Expressions in PHP

Regular expressions (regex) are used to search, match, and manipulate patterns in strings in PHP. Functions like `preg_match()` are commonly used.

Example:

```
<?php
$str = "hello123";

if (preg_match("/[0-9]+/", $str)) {
    echo "Contains numbers";
}
?>
```

17. Explain form handling in PHP with an example.

Form Handling in PHP

Form handling in PHP is used to collect user input from HTML forms and process it on the server using methods like GET or POST.

Example:

HTML Form:

```
<form method="post" action="process.php">
    Name: <input type="text" name="username" />
    <input type="submit" />
</form>
```

PHP (process.php):

```
<?php
```

```
$name = $_POST["username"];  
echo "Hello " . $name;  
?>
```

18. Describe file handling in PHP and explain how to read and write data into files.

File Handling in PHP

File handling in PHP allows creating, reading, writing, and closing files on the server using built-in functions.

Reading a File:

```
<?php  
$file = fopen("test.txt", "r");  
echo fread($file, filesize("test.txt"));  
fclose($file);  
?>
```

Writing to a File:

```
<?php  
$file = fopen("test.txt", "w");  
fwrite($file, "Hello World");  
fclose($file);  
?>
```

19. Explain cookies and session tracking in PHP and discuss their importance in web applications.

Cookies and Session Tracking in PHP

Cookies:

Cookies are small pieces of data stored in the user's browser to remember information.

Example:

```
<?php
setcookie("user", "John", time()+3600);
echo $_COOKIE["user"];
?>
```

Sessions:

Sessions store data on the server and track users using a unique session ID.

Example:

```
<?php
session_start();
$_SESSION["user"] = "John";
echo $_SESSION["user"];
?>
```

Importance:

- Maintain user login state
- Store user preferences
- Enable tracking across pages

20 Explain database connectivity using PHP and MySQL and describe how to perform basic database operations.

Database Connectivity using PHP and MySQL

PHP connects to MySQL using functions like `mysqli_connect()` to interact with databases.

Example (Connection):

```
<?php
$conn = mysqli_connect("localhost", "root", "", "testdb");
if (!$conn) {
    die("Connection failed");
}
?>
```

Basic Database Operations (CRUD):

1. Insert:

```
mysqli_query($conn, "INSERT INTO users(name) VALUES('John')");
```

2. Select:

```
$result = mysqli_query($conn, "SELECT * FROM users");
```

3. Update:

```
mysqli_query($conn, "UPDATE users SET name='Sam' WHERE id=1");
```

4. Delete:

```
mysqli_query($conn, "DELETE FROM users WHERE id=1");
```

21. Explain the overview and features of JavaScript and its role in web development.

Overview of JavaScript

JavaScript is a client-side scripting language used to make web pages interactive and dynamic. It runs in the browser.

Features of JavaScript

- Lightweight and fast
- Interpreted language
- Supports event handling

- Object-based scripting
 - Platform independent
-

Role in Web Development

- Adds interactivity (forms, buttons, validation)
- Manipulates HTML/CSS dynamically
- Handles events (click, submit)
- Used in frontend frameworks

22. Discuss the syntactic characteristics of JavaScript and explain screen output and keyboard input.

JavaScript Syntax Characteristics

- Case-sensitive language
- Statements end with ;
- Variables declared using var, let, const
- Uses {} for blocks

Example:

```
let name = "John";
```

Screen Output

JavaScript displays output using:

- console.log()
- document.write()
- alert()

```
console.log("Hello");  
alert("Hi");
```

Keyboard Input

Input can be taken using prompt().

```
let name = prompt("Enter name");
```

23. Explain control statements in JavaScript with examples.

Control Statements in JavaScript

Control statements manage the flow of execution in a program.

Types:

1. Conditional Statements – if, else, switch

```
let age = 18;  
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

2. Looping Statements – for, while

```
for (let i = 1; i <= 3; i++) {  
  console.log(i);  
}
```

24. Describe arrays and functions in JavaScript with suitable examples.

Arrays in JavaScript

Arrays store multiple values in a single variable.

Example:

```
let arr = [10, 20, 30];  
console.log(arr[0]);
```

Functions in JavaScript

Functions are reusable blocks of code used to perform tasks.

Example:

```
function add(a, b) {  
  return a + b;  
}
```

```
console.log(add(5, 3));
```

25. Explain object creation and modification in JavaScript.

Object Creation in JavaScript

Objects store data in key-value pairs.

Example:

```
let person = {  
  name: "John",  
  age: 20  
};
```

Object Modification

Properties can be added, updated, or deleted.

Example:

```
person.age = 25;    // update  
person.city = "Delhi"; // add  
delete person.name; // delete
```

26. Explain constructors in JavaScript with an example.

Constructors in JavaScript

A constructor is a special function used to create and initialize objects. It is called using the new keyword.

Example:

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```

```
let p1 = new Person("John", 20);  
console.log(p1.name);
```

Explanation:

- this refers to the current object
- new creates a new object instance

27. Describe pattern matching in JavaScript using regular expressions.

Pattern Matching using Regular Expressions in JavaScript

Regular expressions (regex) are used to search and match patterns in strings.

Example:

```
let str = "hello123";  
let pattern = /[0-9]+/;  
  
if (pattern.test(str)) {  
  console.log("Contains numbers");  
}
```

28. Explain the JavaScript execution environment and Document Object Model (DOM).

JavaScript Execution Environment

It is the environment where JavaScript code runs, mainly the **browser**. It includes:

- JavaScript engine (executes code)
 - Call stack and memory
 - Browser APIs
-

Document Object Model (DOM)

DOM represents an HTML document as a tree of objects, allowing JavaScript to access and modify elements dynamically.

Example:

```
document.getElementById("demo").innerHTML = "Hello";
```

29. Explain event handling in JavaScript with examples for button and textbox elements.

Event Handling in JavaScript

Event handling allows JavaScript to respond to user actions like clicks or typing.

Button Example (Click Event):

```
<button onclick="showMsg()">Click</button>
```

```
<script>
function showMsg() {
  alert("Button clicked");
}
</script>
```

Textbox Example (Input Event):

```
<input type="text" onkeyup="showText(this.value)" />
```

```
<script>
function showText(val) {
  console.log(val);
}
</script>
```

30. Discuss the Navigator object and its properties in JavaScript.

Navigator Object in JavaScript

The navigator object provides information about the browser and system.

Common Properties:

- navigator.userAgent → browser details
 - navigator.platform → OS information
 - navigator.language → browser language
 - navigator.onLine → online status
-

Example:

```
console.log(navigator.userAgent);  
console.log(navigator.platform);
```